

Artificial Intelligence

Prof. Chang-Hee Han

2

The mathematical building blocks of neural networks

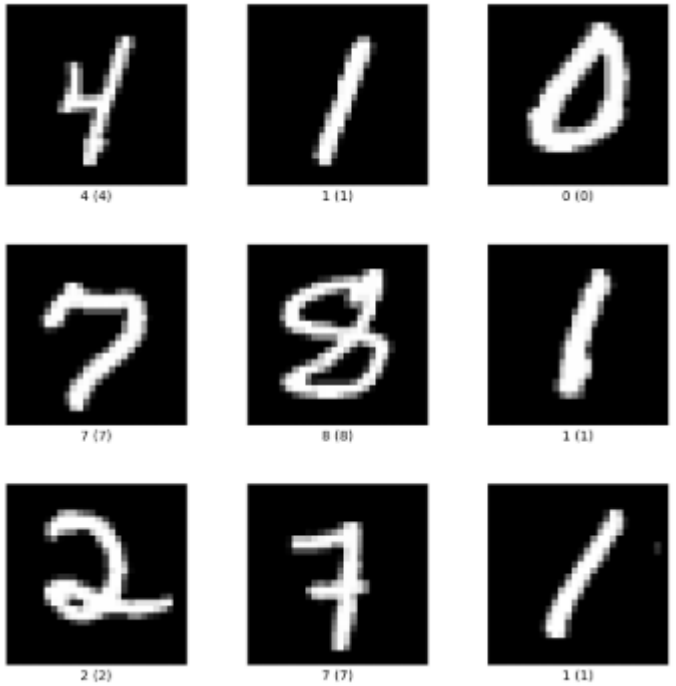
2.0 This chapter covers

- **To understand deep learning, we need some basic math ideas:**
  - Tensors & tensor operations
  - Differentiation
  - Gradient descent
- **Goal of this chapter:**
  - Build intuition, not heavy math
- **Approach:**
  - Start with a practical neural network example
  - Explain each concept step by step
- **Outcome:**
  - Gain an intuitive understanding of the math
  - Be ready to apply it in real deep learning projects

2.1 A first look at a neural network

A first look at a neural network

- Task: Classify handwritten digit images (0–9)
  - Grayscale, 28×28 pixels
- Dataset: MNIST
  - 60,000 training images + 10,000 test images
  - Created in 1980s (NIST)
  - The “Hello World” of deep learning
- Approach:
  - Use Keras neural network to learn classification
  - Don’t worry if steps look confusing → will be explained in detail later



2.1 A first look at a neural network



Figure 2.1 MNIST sample digits

NOTE In machine learning, a *category* in a classification problem is called a *class*. Data points are called *samples*. The class associated with a specific sample is called a *label*.

2.1 A first look at a neural network

▣ A first look at a neural network

- You don't need to try to reproduce this example on your machine just now.
  - ➢ If you wish to, you'll first need to set up a deep learning workspace, which is covered in chapter 3.
- The MNIST dataset comes preloaded in Keras, in the form of a set of four NumPy arrays.

Listing 2.1 Loading the MNIST dataset in Keras

```
from tensorflow.keras.datasets import mnist
(train_images, train_labels), (test_images, test_labels) = mnist.load_data()
```

`train_images` and `train_labels` form the training set, the data that the model will learn from. The model will then be tested on the test set, `test_images` and `test_labels`.

2.1 A first look at a neural network

▣ A first look at a neural network

- The images are encoded as NumPy arrays, and the labels are an array of digits, ranging from 0 to 9.
- The images and labels have a one-to-one correspondence. Let's look at the training data:

```
>>> train_images.shape
(60000, 28, 28)
>>> len(train_labels)
60000
>>> train_labels
array([5, 0, 4, ..., 5, 6, 8], dtype=uint8)
```

2.1 A first look at a neural network

▣ A first look at a neural network

- And here's the test data:

```
>>> test_images.shape
(10000, 28, 28)
>>> len(test_labels)
10000
>>> test_labels
array([7, 2, 1, ..., 4, 5, 6], dtype=uint8)
```

2.1 A first look at a neural network

▣ A first look at a neural network

- The workflow will be as follows:
  - ➢ First, we'll feed the neural network the training data, train_images and train_labels.
  - ➢ The network will then learn to associate images and labels.
  - ➢ Finally, we'll ask the network to produce predictions for test_images, and we'll verify whether these predictions match the labels from test_labels.
- Let's build the network—again, remember that you aren't expected to understand everything about this example yet.

2.1 A first look at a neural network

▣ A first look at a neural network

Listing 2.2 The network architecture

```
from tensorflow import keras
from tensorflow.keras import layers
model = keras.Sequential([
    layers.Dense(512, activation="relu"),
    layers.Dense(10, activation="softmax")
])
```

- The layer is the basic unit of a neural network, acting like a filter that transforms data into more useful representations.
- Deep learning chains many layers together, progressively refining data (progressive data distillation).
- Example: a model with two Dense layers, ending in a 10-way softmax layer that outputs probabilities for digit classes (0–9).

2.1 A first look at a neural network

▣ A first look at a neural network

- Before training, the model must be compiled with:
  - ➢ Optimizer → updates weights to improve performance
  - ➢ Loss function → measures how well the model is doing
  - ➢ Metrics → track progress (e.g., accuracy for classification)

Listing 2.3 The compilation step

```
model.compile(optimizer="rmsprop",
              loss="sparse_categorical_crossentropy",
              metrics=["accuracy"])
```

2.1 A first look at a neural network

▣ A first look at a neural network

- Preprocessing step: reshape and scale data before training.
- Original: shape (60000, 28, 28), type uint8, values [0–255].
- Transformed: shape (60000, 784), type float32, values [0–1].

Listing 2.4 Preparing the image data

```
train_images = train_images.reshape((60000, 28 * 28))
train_images = train_images.astype("float32") / 255
test_images = test_images.reshape((10000, 28 * 28))
test_images = test_images.astype("float32") / 255
```

2.1 A first look at a neural network

▣ A first look at a neural network

- We're now ready to train the model, which in Keras is done via a call to the model's fit() method—we fit the model to its training data.

Listing 2.5 "Fitting" the model

```
>>> model.fit(train_images, train_labels, epochs=5, batch_size=128)
Epoch 1/5
60000/60000 [==============================] - 5s - loss: 0.2524 - acc: 0.9273
Epoch 2/5
51328/60000 [========================>.....] - ETA: 1s - loss: 0.1035 - acc: 0.9692
```

- Two quantities are displayed during training: the loss of the model over the training data, and the accuracy of the model over the training data. We quickly reach an accuracy of 0.989 (98.9%) on the training data.

2.1 A first look at a neural network

▣ A first look at a neural network

- Now that we have a trained model, we can use it to predict class probabilities for new digits—images that weren't part of the training data, like those from the test set.

Listing 2.6 Using the model to make predictions

```
>>> test_digits = test_images[0:10]
>>> predictions = model.predict(test_digits)
>>> predictions[0]
array([1.0726176e-10, 1.6918376e-10, 6.1314843e-08, 8.4106023e-06,
       2.9967067e-11, 3.0331331e-09, 8.3651971e-14, 9.9999106e-01,
       2.6657624e-08, 3.8127661e-07], dtype=float32)
```

- Each number of index i in that array corresponds to the probability that digit image test_digits[0] belongs to class i.

2.1 A first look at a neural network

▣ A first look at a neural network

- This first test digit has the highest probability score (0.99999106, almost 1) at index 7, so according to our model, it must be a 7:

```
>>> predictions[0].argmax()
7
>>> predictions[0][7]
0.99999106

>>> test_labels[0]
7
```

2.1 A first look at a neural network

A first look at a neural network

- On average, how good is our model at classifying such never-before-seen digits?
- Let's check by computing average accuracy over the entire test set.

Listing 2.7 Evaluating the model on new data

```
>>> test_loss, test_acc = model.evaluate(test_images, test_labels)
>>> print(f"test_acc: {test_acc}")
test_acc: 0.9785
```

2.1 A first look at a neural network

▣ A first look at a neural network

- Result:
  - ➢ Test accuracy = 97.8%, lower than training accuracy = 98.9% → shows overfitting.
- Overfitting
  - ➢ models perform worse on new data than on training data (covered in Chapter 3).
- This first example showed how to build/train a digit classifier in <15 lines of Python.
- Next chapters will explain the details:
  - ➢ tensors, tensor operations, and gradient descent.

2.2 Data representations for neural networks

▣ Data representations for neural networks

- In the previous example, we started from data stored in multidimensional NumPy arrays, also called tensors.
- In general, all current machine learning systems use tensors as their basic data structure.
- Tensors are fundamental to the field—so fundamental that TensorFlow was named after them.
- **So what's a tensor?**
  - ➢ At its core, a tensor is a container for data—usually numerical data.
  - ➢ So, it's a container for numbers.
- You may be already familiar with matrices, which are rank-2 tensors:
  - ➢ tensors are a generalization of matrices to an arbitrary number of dimensions (note that in the context of tensors, a dimension is often called an axis).

2.2.1 Scalars (rank-0 tensors)

▣ Scalars (rank-0 tensors)

- A tensor that contains only one number is called a scalar (or scalar tensor, or rank-0 tensor, or 0D tensor).
- In NumPy, a float32 or float64 number is a scalar tensor (or scalar array).
- You can display the number of axes of a NumPy tensor via the ndim attribute; a scalar tensor has 0 axes (ndim == 0).
- The number of axes of a tensor is also called its rank.
- Here's a NumPy scalar:

```
>>> import numpy as np
>>> x = np.array(12)
>>> x
array(12)
>>> x.ndim
0
```

2.2.2 Vectors (rank-1 tensors)

▣ Vectors (rank-1 tensors)

- An array of numbers is called a vector, or rank-1 tensor, or 1D tensor.
- A rank-1 tensor is said to have exactly one axis.
- Following is a NumPy vector:

```
>>> x = np.array([12, 3, 6, 14, 7])
>>> x
array([12, 3, 6, 14, 7])
>>> x.ndim
1
```

- This vector has five entries and so is called a 5-dimensional vector.
- Don’t confuse a 5D vector with a 5D tensor!

2.2.3 Matrices (rank-2 tensors)

▣ Matrices (rank-2 tensors)

- An array of vectors is a matrix, or rank-2 tensor, or 2D tensor.
- A matrix has two axes (often referred to as rows and columns).
- You can visually interpret a matrix as a rectangular grid of numbers.
- This is a NumPy matrix:

```
>>> x = np.array([[5, 78, 2, 34, 0],
                  [6, 79, 3, 35, 1],
                  [7, 80, 4, 36, 2]])
>>> x.ndim
2
```

- Rows come from the **first axis** and columns from the **second axis** of a tensor.
- [5, 78, 2, 34, 0] is the first row of x, and [5, 6, 7] is the first column.

2.2.4 Rank-3 and higher-rank tensors

▣ Rank-3 and higher-rank tensors

- If you pack such matrices in a new array, you obtain a rank-3 tensor (or 3D tensor), which you can visually interpret as a cube of numbers.
- Following is a NumPy rank-3 tensor:

```
>>> x = np.array([[[5, 78, 2, 34, 0],
                   [6, 79, 3, 35, 1],
                   [7, 80, 4, 36, 2]],
                  [[5, 78, 2, 34, 0],
                   [6, 79, 3, 35, 1],
                   [7, 80, 4, 36, 2]],
                  [[5, 78, 2, 34, 0],
                   [6, 79, 3, 35, 1],
                   [7, 80, 4, 36, 2]]])
>>> x.ndim
3
```

2.2.5 Key attributes

Number of axes (rank)

- Rank = number of dimensions (axes)
- Example: matrix has 2 axes, rank-3 tensor has 3 axes

Shape

- Tuple of integers showing size along each axis
- Example:
  - Matrix: shape (3, 5)
  - Rank-3 tensor: shape (3, 3, 5)
  - Vector: (5,)
  - Scalar: ()

Data type (dtype)

- Type of numbers stored in the tensor
- Examples: float16, float32, float64, uint8, string

2.2.6 Manipulating tensors in NumPy

▣ Manipulating tensors in NumPy

- Selecting specific elements in a tensor is called **tensor slicing.**
- The following example selects digits #10 to #100 (#100 isn't included) and puts them in an array of shape (90, 28, 28):

```
>>> my_slice = train_images[10:100]
>>> my_slice.shape
(90, 28, 28)
```

2.2.6 Manipulating tensors in NumPy

▣ Manipulating tensors in NumPy

- It's equivalent to this more detailed notation, which specifies a start index and stop index for the slice along each tensor axis.
- Note that : is equivalent to selecting the entire axis:

```
>>> my_slice = train_images[10:100, :, :]
>>> my_slice.shape
(90, 28, 28)
>>> my_slice = train_images[10:100, 0:28, 0:28]
>>> my_slice.shape
(90, 28, 28)
```

Equivalent to the previous example

Also equivalent to the previous example

2.2.6 Manipulating tensors in NumPy

▣ Manipulating tensors in NumPy

- In general, you may select slices between any two indices along each tensor axis.
- For instance, in order to select 14 x 14 pixels in the bottom-right corner of all images, you would do this:

```
my_slice = train_images[:, 14:, 14:]
```

- It's also possible to use negative indices. Much like negative indices in Python lists, they indicate a position relative to the end of the current axis. In order to crop the images to patches of 14 x 14 pixels centered in the middle, you'd do this:

```
my_slice = train_images[:, 7:-7, 7:-7]
```

2.2.7 The notion of data batches

▣ Manipulating tensors in NumPy

- In general, the first axis (axis 0, because indexing starts at 0) in all data tensors you'll come across in deep learning will be the **samples axis** (sometimes called the samples dimension). In the MNIST example, "samples" are images of digits.
- In addition, deep learning models don't process an entire dataset at once; rather, they break the data into small batches. Concretely, here's one batch of our MNIST digits, with a **batch** size of 128:

First batch

```
batch = train_images[:128]
```

Next batch

```
batch = train_images[128:256]
```

N^{th} batch

```
n = 3
batch = train_images[128 * n:128 * (n + 1)]
```

2.2.8 Real-world examples of data tensors

▣ Real-world examples of data tensors

- Let's make data tensors more concrete with a few examples similar to what you'll encounter later. The data you'll manipulate will almost always fall into one of the following categories:
  - Vector data (Rank-2) → shape (samples, features)
  - Timeseries/sequence data (Rank-3) → shape (samples, timesteps, features)
  - Images (Rank-4) → shape (samples, height, width, channels)
  - Video (Rank-5) → shape (samples, frames, height, width, channels)
- it's possible to add tensors, multiply tensors, and so on.

The gears of neural networks: Tensor operations

- In our initial example, we built our model by stacking Dense layers on top of each other. A Keras layer instance looks like this:

```
keras.layers.Dense(512, activation="relu")
```

- This layer can be interpreted as a function, which takes as input a matrix and returns another matrix—a new representation for the input tensor. Specifically, the function is as follows (where W is a matrix and b is a vector, both attributes of the layer):

```
output = relu(dot(input, W) + b)
```

▣ The gears of neural networks: Tensor operations

```
output = relu(dot(input, W) + b)
```

- Let's unpack this. We have three tensor operations here:
  - A dot product (dot) between the input tensor and a tensor named W
  - An addition (+) between the resulting matrix and a vector b
  - A relu operation: relu(x) is max(x, 0); "relu" stands for "rectified linear unit"

2.3 The gears of neural networks: Tensor operations

2.3.1 Element-wise operations

- ReLU and addition are element-wise operations, applied independently to each tensor entry. These operations can be efficiently parallelized (vectorized), though a naive Python version would use a simple for loop.

```
def naive_relu(x):
    assert len(x.shape) == 2

    x = x.copy()
    for i in range(x.shape[0]):
        for j in range(x.shape[1]):
            x[i, j] = max(x[i, j], 0)
    return x
```

x is a rank-2 NumPy tensor.

Avoid overwriting the input tensor.

2.3 The gears of neural networks: Tensor operations

2.3.1 Element-wise operations

- You could do the same for addition:

```
def naive_add(x, y):
    assert len(x.shape) == 2
    assert x.shape == y.shape
    x = x.copy()
    for i in range(x.shape[0]):
        for j in range(x.shape[1]):
            x[i, j] += y[i, j]
    return x
```

x and y are rank-2 NumPy tensors.

Avoid overwriting the input tensor.

2.3 The gears of neural networks: Tensor operations

2.3.2 Broadcasting

- What happens with addition when the shapes of the two tensors being added differ?
- The smaller tensor will be broadcast to match the shape of the larger tensor.
- Broadcasting consists of two steps:
  ① Axes (called broadcast axes) are added to the smaller tensor to match the ndim of the larger tensor.
  ② The smaller tensor is repeated alongside these new axes to match the full shape of the larger tensor.

2.3 The gears of neural networks: Tensor operations

2.3.2 Broadcasting

- Let's look at a concrete example. Consider X with shape (32, 10) and y with shape (10,):

```
import numpy as np
X = np.random.random((32, 10))
y = np.random.random((10,))
```

X is a random matrix with shape (32, 10).

y is a random vector with shape (10,).

2.3 The gears of neural networks: Tensor operations

2.3.2 Broadcasting

- First, we add an empty first axis to y, whose shape becomes (1, 10):

```
y = np.expand_dims(y, axis=0)
```

The shape of y is now (1, 10).

- Then, we repeat y 32 times alongside this new axis, so that we end up with a tensor Y with shape (32, 10), where Y[i, :] == y for i in range(0, 32):

```
Y = np.concatenate([y] * 32, axis=0)
```

Repeat y 32 times along axis 0 to obtain Y, which has shape (32, 10).

- Broadcasting lets you perform element-wise operations on two tensors by automatically expanding the smaller tensor to match the larger tensor's shape along missing axes.

2.3 The gears of neural networks: Tensor operations

2.3.3 Tensor product

- The tensor product, or dot product (not to be confused with an element-wise product, the * operator), is one of the most common, most useful tensor operations.
- In NumPy, a tensor product is done using the np.dot function (because the mathematical notation for tensor product is usually a dot):

```
x = np.random.random((32,))
y = np.random.random((32,))
z = np.dot(x, y)
```

- In mathematical notation, you'd note the operation with a dot (•):

$$z = x \cdot y$$

2.3 The gears of neural networks: Tensor operations

2.3.3 Tensor product

- To understand dot-product shape compatibility, it helps to visualize the input and output tensors by aligning them as shown in figure 2.5.

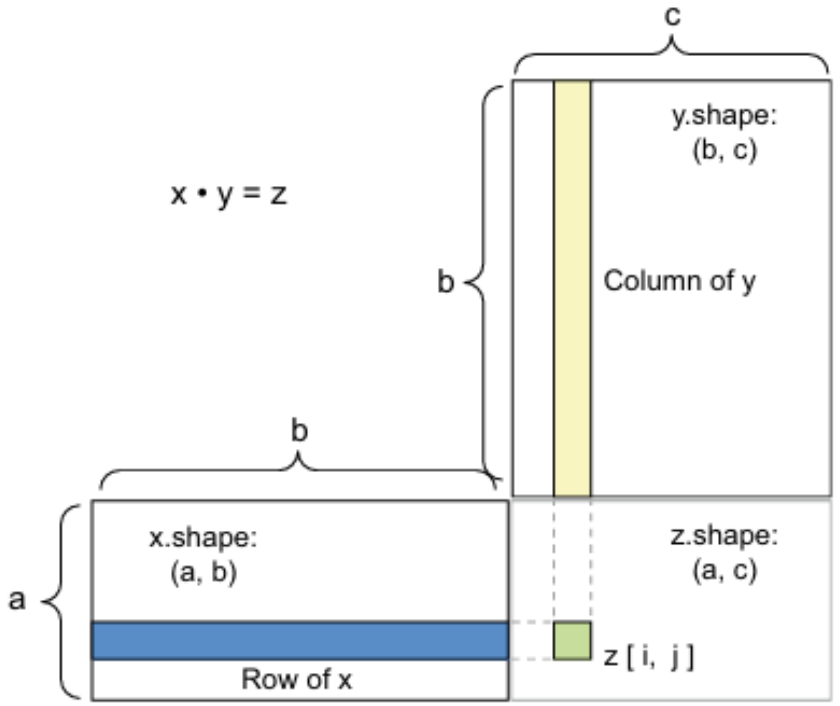


Figure 2.5 Matrix dot-product box diagram

2.3 The gears of neural networks: Tensor operations

2.3.4 Tensor reshaping

- Reshaping a tensor means rearranging its rows and columns to match a target shape.
- Naturally, the reshaped tensor has the same total number of coefficients as the initial tensor.

```
>>> x = np.array([[0., 1.],
                  [2., 3.],
                  [4., 5.]])
>>> x.shape
(3, 2)
>>> x = x.reshape((6, 1))
>>> x
array([[ 0.],
       [ 1.],
       [ 2.],
       [ 3.],
       [ 4.],
       [ 5.]])
```

2.4 The engine of neural networks: Gradient-based optimization

▣ Training Loop in Neural Networks

- Layer operation: `output = relu(dot(input, W) + b)`
  - W, b = trainable parameters (weights, bias)
  - Contain the knowledge learned from training
- Initialization:
  - Weights start as small random values (random initialization)
  - Initial outputs are meaningless, but serve as a starting point

2.4 The engine of neural networks: Gradient-based optimization

▣ Training Loop in Neural Networks

- Training = Gradual adjustment of weights
  - ➢ Based on feedback (loss function)
- Training loop steps:
  - ① Draw a batch of training samples (x, y_true)
  - ② Run forward pass → predictions (y_pred)
  - ③ Compute loss = mismatch between y_pred and y_true
  - ④ Update weights slightly to reduce loss

2.4 The engine of neural networks: Gradient-based optimization

▣ Gradient Descent – The Engine of Neural Networks

- Goal: Update weights to reduce loss (difference between prediction y_pred and target y_true)
- Naive approach:
  - ➢ Change one weight at a time and test → very inefficient (millions of coefficients)
- Better approach: Gradient Descent
  - ➢ Functions used in models are differentiable
  - ➢ A small weight change → small, predictable change in loss
  - ➢ Compute the gradient = direction of steepest loss decrease
  - ➢ Update all weights together, moving them to reduce loss
- Key idea: Gradient descent makes training efficient and scalable

2.4 The engine of neural networks: Gradient-based optimization

2.4.1 What's a derivative?

- Consider a continuous, smooth function f(x) = y, mapping a number, x, to a new number, y. We can use the function in figure 2.15 as an example.

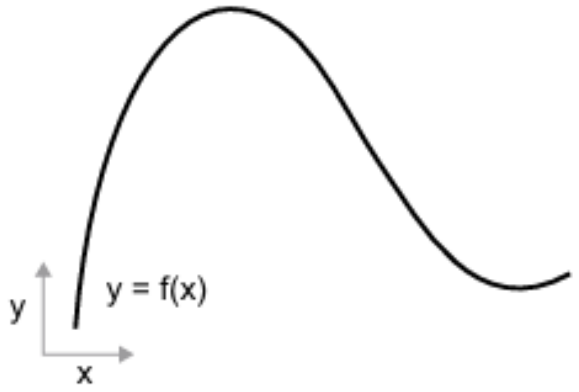


Figure 2.15 A continuous, smooth function

- Because the function is continuous, a small change in x can only result in a small change in y—that's the intuition behind continuity.

2.4 The engine of neural networks: Gradient-based optimization

2.4.1 What's a derivative?

- Let's say you increase x by a small factor, epsilon_x: this results in a small epsilon_y change to y, as shown in figure 2.16.

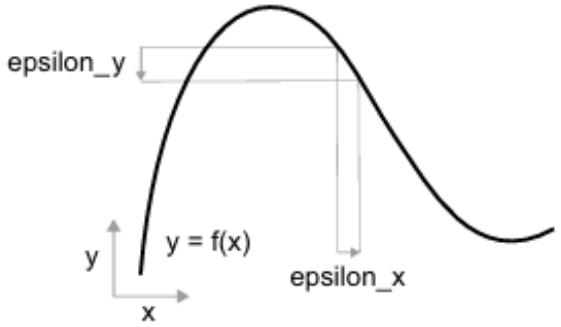


Figure 2.16 With a continuous function, a small change in x results in a small change in y.

- In addition, because the function is smooth (its curve doesn't have any abrupt angles), when epsilon_x is small enough, around a certain point p, it's possible to approximate f as a linear function of slope a, so that epsilon_y becomes a * epsilon_x:

```
f(x + epsilon_x) = y + a * epsilon_x
```

2.4.1 What's a derivative?

- The slope a is called the derivative of f in p. If a is negative, it means a small increase in x around p will result in a decrease of f(x) (as shown in figure 2.17), and if a is positive, a small increase in x will result in an increase of f(x). Further, the absolute value of a (the magnitude of the derivative) tells you how quickly this increase or decrease will happen.

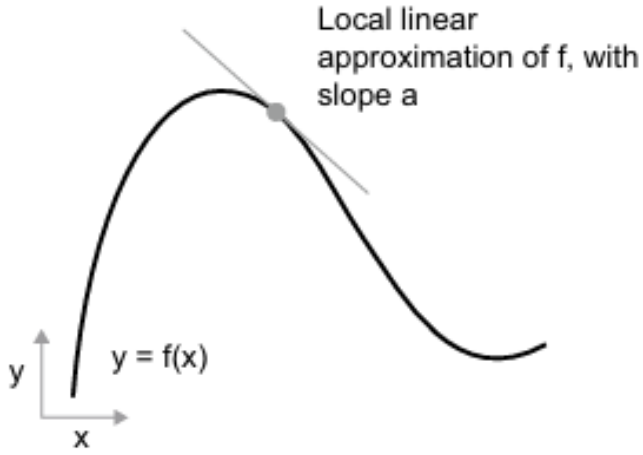


Figure 2.17 Derivative of f in p

2.4 The engine of neural networks: Gradient-based optimization

▣ 2.4.1 What's a derivative?

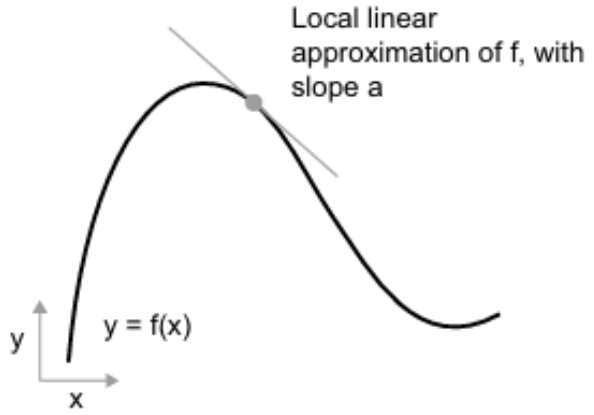


Figure 2.17 Derivative of f in p

- A derivative f′(x) gives the slope of a function at each point.
- Examples: derivative of cos(x) = –sin(x), derivative of a·x = a.
- Derivatives are powerful for optimization: they show how f(x) changes as x changes.
- To minimize f(x), move x slightly in the opposite direction of the derivative.

2.4.2 Derivative of a tensor operation: The gradient

- Derivative (scalar → scalar)
  - ✓ Slope of curve in 2D
- Vector & tensor functions
  - ✓ (x, y) → z → surface in 3D
  - ✓ Can extend to matrices, rank-3 tensors, etc.
- Gradient = derivative generalized to tensor functions
  - ✓ Describes how output changes when inputs vary
  - ✓ Represents slope/curvature of multidimensional surface
- Machine Learning Example
  - ✓ Input vector x (sample)
  - ✓ Matrix W (weights)
  - ✓ Target y_true
  - ✓ Loss function measures gap between prediction & target

2.4.2 Derivative of a tensor operation: The gradient

- You can use W to compute a target candidate y_pred, and then compute the loss, or mismatch, between the target candidate y_pred and the target y_true:

```
y_pred = dot(W, x)
loss_value = loss(y_pred, y_true)
```

We use the model weights, W, to make a prediction for x.

We estimate how far off the prediction was.

- Now we'd like to use gradients to figure out how to update W so as to make loss_value smaller. How do we do that?

2.4 The engine of neural networks: Gradient-based optimization

2.4.2 Derivative of a tensor operation: The gradient

- Given fixed inputs x and y_true, the preceding operations can be interpreted as a function mapping values of W (the model's weights) to loss values:

```
loss_value = f(W)
```

f describes the curve (or high-dimensional surface) formed by loss values when W varies.

- Let's say the current value of W is W0. Then the derivative of f at the point W0 is a tensor grad(loss_value, W0), with the same shape as W, where each coefficient grad(loss_value, W0)[i, j] indicates the direction and magnitude of the change in loss_value you observe when modifying W0[i, j]. That tensor grad(loss_value, W0) is the gradient of the function f(W) = loss_value in W0, also called "gradient of loss_value with respect to W around W0."

2.4.2 Derivative of a tensor operation: The gradient

- Gradient meaning
  - grad(loss, W0) = direction & slope of steepest ascent of the loss near W0
  - Each partial derivative = slope in one direction
- How to reduce loss
  - Move weights in the opposite direction of the gradient
  - Update rule:

```
W1 = W0 - step * grad(f(W0), W0)
```

2.4 The engine of neural networks: Gradient-based optimization

▣ 2.4.3 Stochastic gradient descent

- Goal: Find weight values that minimize the loss function
- Analytic solution is impossible for large neural networks (millions of parameters)
- **SGD Algorithm (mini-batch version):**

① Draw a batch of training samples (x, y_true)

② Run forward pass → predictions (y_pred)

③ Compute loss between y_pred and y_true

④ Compute gradient of loss (backward pass)

⑤ Update weights in the opposite direction of gradient

- Example: W -= learning_rate * gradient

➢ Learning rate controls the update step size (speed of learning)

➢ "Stochastic" = each batch is drawn at random

▣ 2.4.3 Stochastic gradient descent

- Mini-batch SGD trains a model by using small random batches of data and updating the parameters step by step.

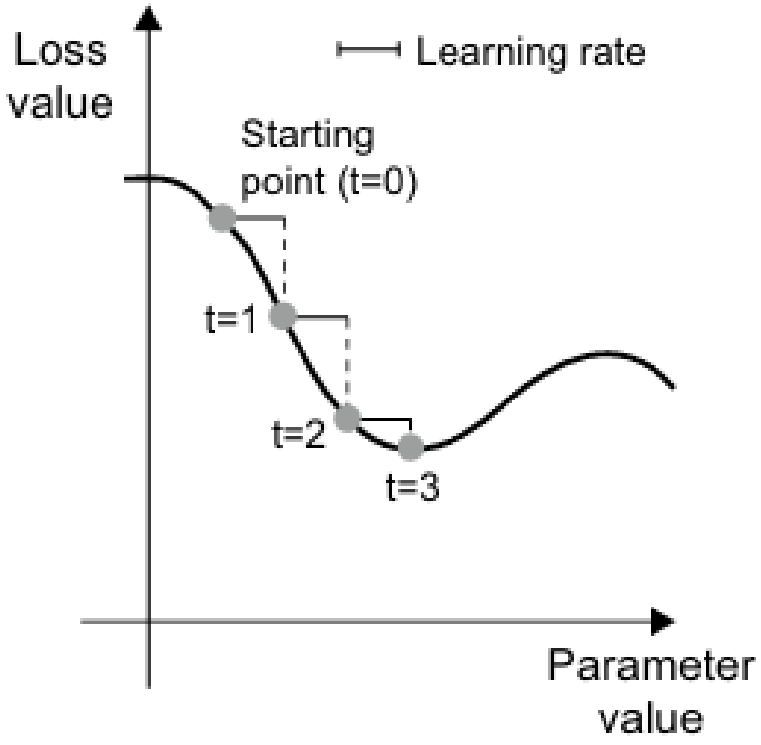


Figure 2.18 SGD down a 1D loss curve (one learnable parameter)

2.4 The engine of neural networks: Gradient-based optimization

2.4.3 Stochastic gradient descent

- Choosing the right learning rate is crucial:
  - Too small → very slow, may get stuck in local minima
  - Too large → updates become unstable/random
- Variants of gradient descent:
  - True SGD → single sample per step
  - Batch GD → all data per step (accurate but expensive)
  - Mini-batch SGD → practical compromise
- In practice:
  - Neural networks have tens of thousands to millions of parameters
  - We can visualize in 1D/2D for intuition, but real training happens in very high-dimensional spaces
  - Low-dimensional intuition is helpful, but can be misleading

2.4 The engine of neural networks: Gradient-based optimization

▣ 2.4.3 Stochastic gradient descent

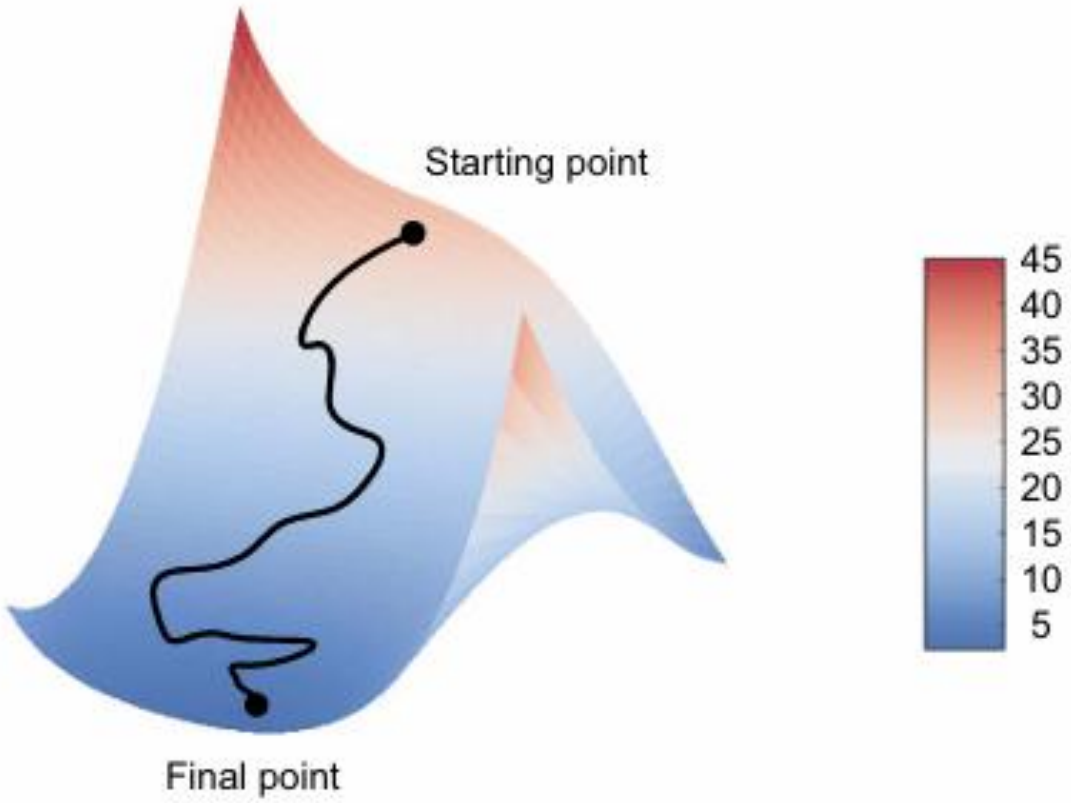


Figure 2.19 Gradient descent down a 2D loss surface (two learnable parameters)

2.4.3 Stochastic gradient descent

- Problem: With a small learning rate, SGD can get stuck in local minima.
- Solution: Momentum
- Inspired by physics → like a ball rolling down a curve
- Uses both current gradient (slope) and previous updates (velocity)
- Helps the optimizer escape local minima and reach the global minimum.

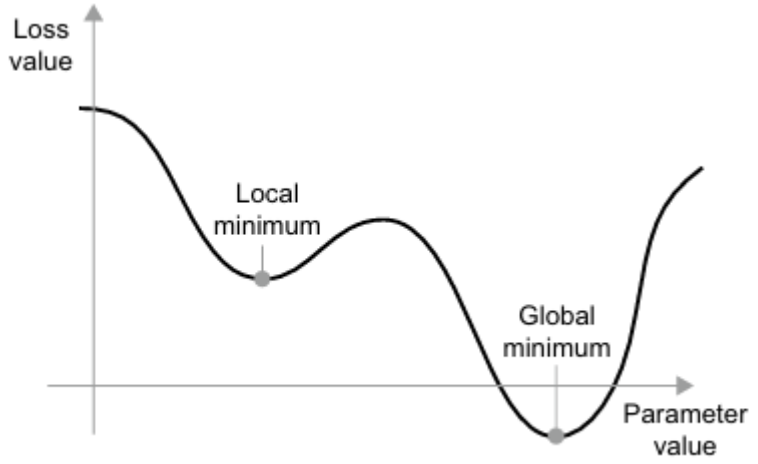


Figure 2.20 A local minimum and a global minimum

▣ 2.4.4 Chaining derivatives: The Backpropagation algorithm

- Gradient Computation & Backpropagation
  - ➢ Problem
    - We assumed differentiable functions → easy gradients
    - But real models = complex expressions
    - Example: two-layer model → need gradient of loss w.r.t. weights
  - ➢ Key Question
    - How do we compute gradients in practice?
  - ➢ Solution
    - **Backpropagation Algorithm**
      - ✓ Efficient method to calculate gradients
      - ✓ Works layer by layer using chain rule

▣ 2.4.4 Chaining derivatives: The Backpropagation algorithm

- **THE CHAIN RULE**

  - ➢ Backpropagation uses the derivatives of simple operations (like addition, ReLU, dot product) to compute gradients of complex networks.
    - Neural networks = many chained tensor operations
    - Each operation has a simple, known derivative
    - Example: model with parameters (W1, b1, W2, b2) → built from dot, ReLU, softmax, +, and loss, all differentiable

```
loss_value = loss(y_true, softmax(dot(relu(dot(inputs, W1) + b1), W2) + b2))
```

  - ➢ Backprop makes gradient computation easy by combining simple derivatives.

▣ 2.4.4 Chaining derivatives: The Backpropagation algorithm

- **THE CHAIN RULE**
  - ➢ Calculus tells us that such a chain of functions can be derived using the following identity, called the chain rule.
  - ➢ Consider two functions f and g, as well as the composed function fg such that

```
fg(x) == f(g(x)):
```

```
def fg(x):
    x1 = g(x)
    y = f(x1)
    return y
```

  - ➢ Then the chain rule states that grad(y, x) == grad(y, x1) * grad(x1, x).

▣ 2.4.4 Chaining derivatives: The Backpropagation algorithm

- **THE CHAIN RULE**

```
def fghj(x):
    x1 = j(x)
    x2 = h(x1)
    x3 = g(x2)
    y = f(x3)
    return y

grad(y, x) == (grad(y, x3) * grad(x3, x2) *
               grad(x2, x1) * grad(x1, x))
```

➢ This enables you to compute the derivative of fg as long as you know the derivatives of f and g.

▣ 2.4.4 Chaining derivatives: The Backpropagation algorithm

- **Computation Graphs & Backpropagation**
  - ➢ A computation graph is a directed acyclic graph of tensor operations.
  - ➢ It represents a model as nodes (operations) and edges (data flow).
  - ➢ Used in frameworks like TensorFlow → treats computation as data.
  - ➢ Benefits:
    - Easy to transform computations (e.g., distributed version).
    - Automatic gradient generation (for backpropagation).
    - More flexible than just writing code as text.

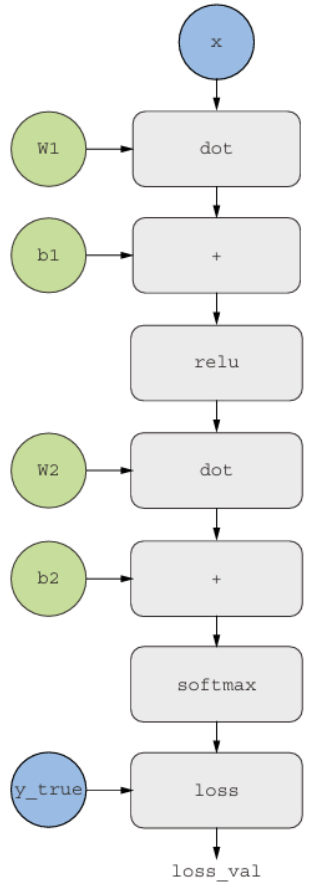


Figure 2.21 The computation graph representation of our two-layer model

2.4.4 The Backpropagation algorithm

- **Basic Backpropagation Example**
  - Let's set concrete values for the "input nodes" in the graph, that is to say, the input x, the target y_true, w, and b.
  - We'll propagate these values to all nodes in the graph, from top to bottom, until we reach loss_val. This is the forward pass (see figure 2.23).

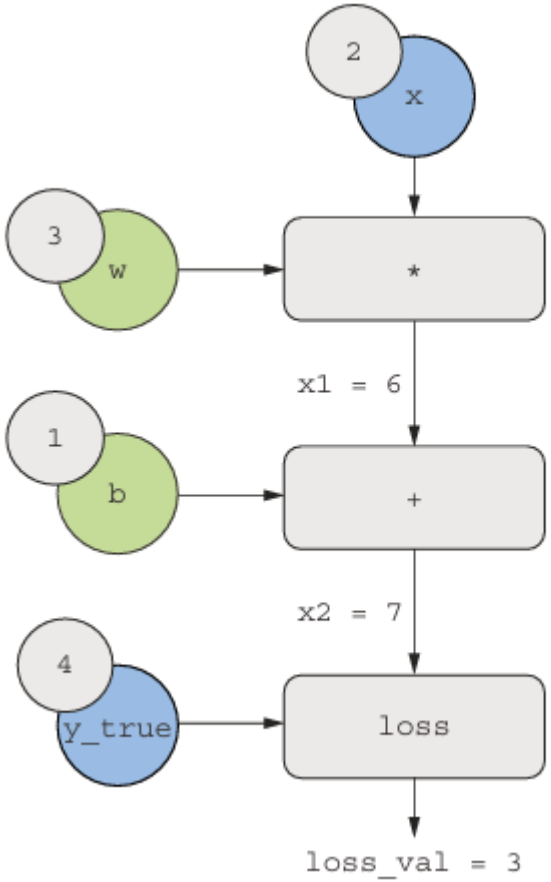


Figure 2.23

▣ 2.4.4 The Backpropagation algorithm

- **Basic Backpropagation Example**
  - ➢ Now let's "reverse" the graph: for each edge in the graph going from A to B, we will create an opposite edge from B to A, and ask, how much does B vary when A varies?
  - ➢ That is to say, what is grad(B, A)? We'll annotate each inverted edge with this value. This backward graph represents the backward pass (see figure 2.24).

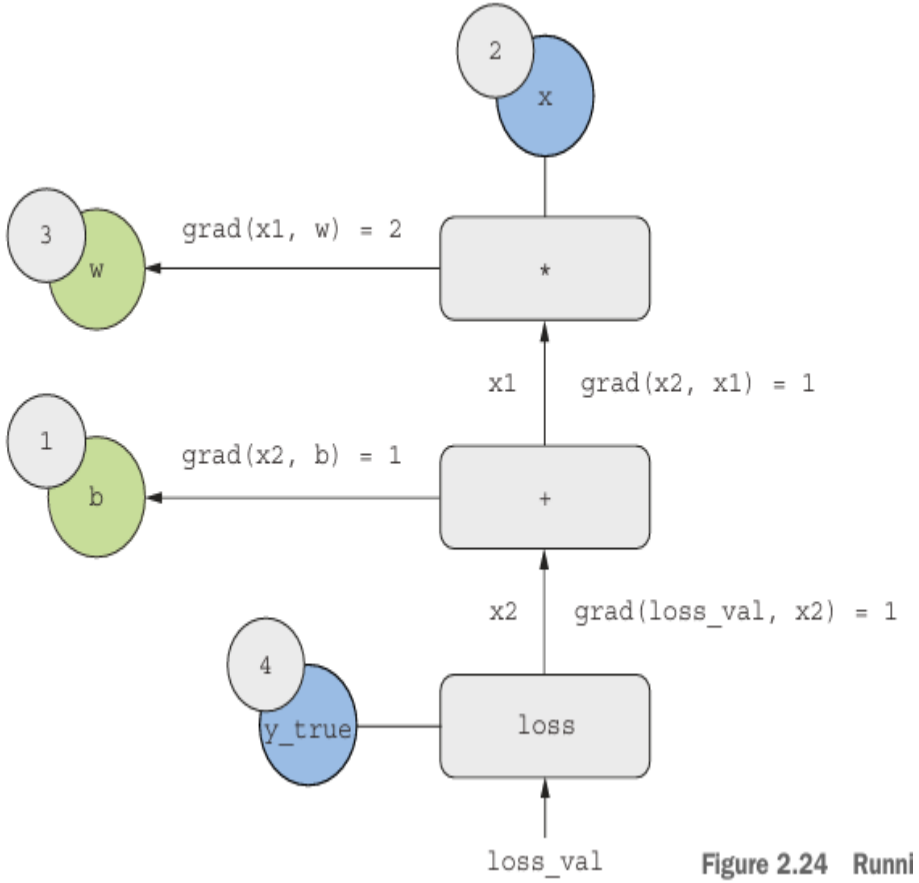


Figure 2.24 Runni

2.4.4 The Backpropagation algorithm

- **Basic Backpropagation Example**
  - What the chain rule says about this backward graph is that you can obtain the derivative of a node with respect to another node by multiplying the derivatives for each edge along the path linking the two nodes.
  - For instance,
    - grad(loss_val, w) = grad(loss_val, x2) * grad(x2, x1) * grad(x1, w)
    - (see figure 2.25).

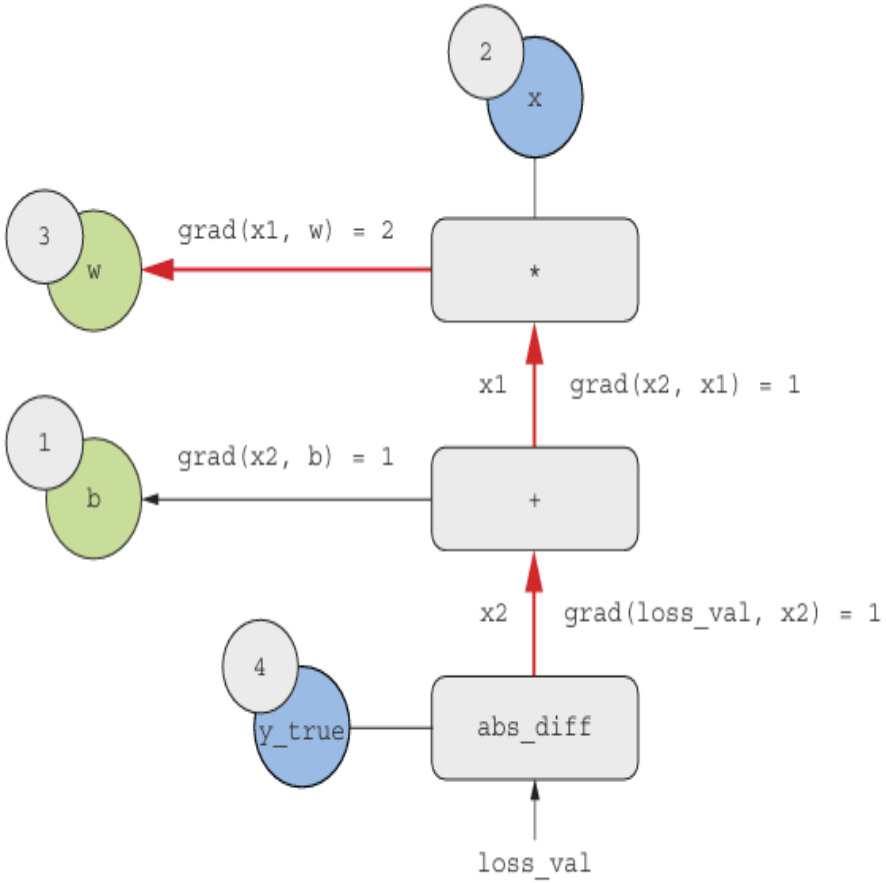


Figure 2.25 Path from loss_val to w in the backward graph

▣ 2.4.4 The Backpropagation algorithm

- **Backpropagation & Automatic Differentiation**
  - ➢ Backpropagation = applying the chain rule on a computation graph.
  - ➢ Process:
    - start from final loss → move backward → compute each parameter's contribution.
  - ➢ Name comes from "back propagating" the loss.
  - ➢ Today, frameworks (e.g., TensorFlow) use automatic differentiation with computation graphs.
  - ➢ This lets us get gradients automatically, just by writing the forward pass.
  - ➢ In the past, gradients had to be written by hand — now, tools handle it for us.



Q & A

Prof. Chang-Hee Han